

Laboratorio #5 - ALE y Space Invaders

CC3092 - Deep Learning y Sistemas Inteligentes

Repositorio: github.com/jlope384/Laboratorio-4-Reinforcement-Learning-101

1. El Arcade Learning Environment (ALE)

Que es ALE y su relacion con Stella y Atari 2600

ALE expone los juegos originales de Atari 2600 como entornos de RL con una interfaz programatica (reset/step/reward), resolviendo el problema de que antes cada quien tenia que integrar el emulador y definir a mano la recompensa y el fin de episodio para cada juego. Por dentro, ALE no reimplementa la consola: usa Stella, un emulador de Atari 2600 de codigo abierto, como motor de emulacion, y anade encima la capa de entorno de RL (extraer el frame, leer la RAM, calcular reward a partir del puntaje, detectar game over). Las ROMs que ejecuta son los cartuchos originales tal cual corrian en el hardware de 1977.

Variantes de un mismo juego y parametros clave

Ademas de la variante estandar en imagen RGB (ALE/SpaceInvaders-v5), el sufijo -ram cambia la observacion a los 128 bytes de memoria de la consola. frameskip controla cuantos frames del emulador se repiten por cada step() (aleatorio entre 2 y 4 por defecto en la v5). repeat_action_probability (0.25 por defecto) es la probabilidad de que se repita la accion anterior en vez de la elegida (sticky actions), agregado para evitar que un agente memorice la secuencia exacta de acciones sin generalizar, ya que el emulador es determinista. full_action_space decide si se exponen las 18 acciones del joystick completo o solo el subconjunto que tiene efecto en ese juego en particular (6 en Space Invaders).

Frame skipping y Space Invaders

El frame skipping importa porque entre frames consecutivos (a 60 FPS) casi no cambia nada: mantener la misma accion varios frames reduce el costo de simulacion y acorta el horizonte temporal efectivo, lo que facilita el aprendizaje del credito temporal. En Space Invaders el jugador mueve un canon horizontal y dispara contra una formacion de invasores que avanza y baja; el objetivo es destruirlos antes de que lleguen abajo o de perder todas las vidas. La recompensa del entorno es, paso a paso, el incremento del puntaje interno del juego (que crece al destruir invasores o la nave bonus), sin señal alguna por sobrevivir.

2. Espacios de observacion y accion en entornos Atari

El espacio de observacion por defecto de ALE/SpaceInvaders-v5 es Box(0, 255, (210, 160, 3), uint8): el frame RGB completo, 100,800 valores por observacion. Comparado con CartPole-v1 (Box(4,), 4 numeros ya resumidos como cantidades fisicas), la diferencia es enorme: en Space Invaders el agente recibe pixeles crudos y necesita algun tipo de procesamiento de imagen (en la practica, una CNN) para extraer que es la nave, los invasores y las balas, con un espacio de entrada varios ordenes de magnitud mas grande.

La variante obs_type='ram' (ALE/SpaceInvaders-ram-v5) expone directamente los 128 bytes de memoria de la consola como Box(0, 255, (128,), uint8). Se preferiria sobre la imagen cuando se busca reducir muchisimo el costo computacional (128 valores contra 100,800) o para hacer ingenieria de features a mano leyendo bytes especificos, aunque su interpretacion no es estandar entre juegos y se aleja del escenario realista de aprender directamente de pixeles.

El espacio de accion de Space Invaders es Discrete(6): NOOP (no hacer nada), FIRE (disparar), RIGHT/LEFT (moverse) y RIGHTFIRE/LEFTFIRE (moverse y disparar a la vez). Con full_action_space=True se expondrían las 18 acciones del joystick completo, pero las adicionales serian redundantes para este juego.

AtariPreprocessing aplica en conjunto: escala de grises, redimensionamiento a 84x84, frame skipping interno (con maximo entre los ultimos dos frames para evitar parpadeos de sprites) y recorte opcional de recompensa. FrameStackObservation apila las ultimas k observaciones (tipicamente 4) en un solo tensor; se usa junto con AtariPreprocessing porque un frame estatico no contiene informacion de movimiento, y apilar frames da la senal temporal minima para inferir velocidades y direcciones sin arquitecturas recurrentes.

3. Modulo de desarrollo: funciones para interactuar con ALE

Las funciones se implementaron en ale_module.py, en la raiz del repositorio (no solo dentro del notebook), para que se puedan reutilizar en laboratorios o el proyecto futuro sin copiar codigo. crear_entorno(nombre_entorno, video_folder=None, episode_trigger=None, name_prefix, **make_kwargs) crea el entorno con gym.make y, si se especifica video_folder, fuerza render_mode='rgb_array' y lo envuelve con gymnasium.wrappers.RecordVideo; funciona igual para Space Invaders que para cualquier otro entorno de Gymnasium. agente_aleatorio(observation, env) retorna env.action_space.sample() como baseline. agente_regla_simple(observation, env) sigue un ciclo fijo FIRE-RIGHT-FIRE-LEFT (regla no aprendida). ejecutar_episodio(env, funcion_agente, max_steps, seed) corre un episodio completo hasta terminated/truncated o max_steps, y retorna (steps, return). generar_video_agente(...) combina todo: crea el entorno con grabacion, corre n_episodios, cierra el entorno (env.close(), indispensable para que el .mp4 se escriba a disco) y retorna las rutas de los videos junto con las metricas de cada episodio.

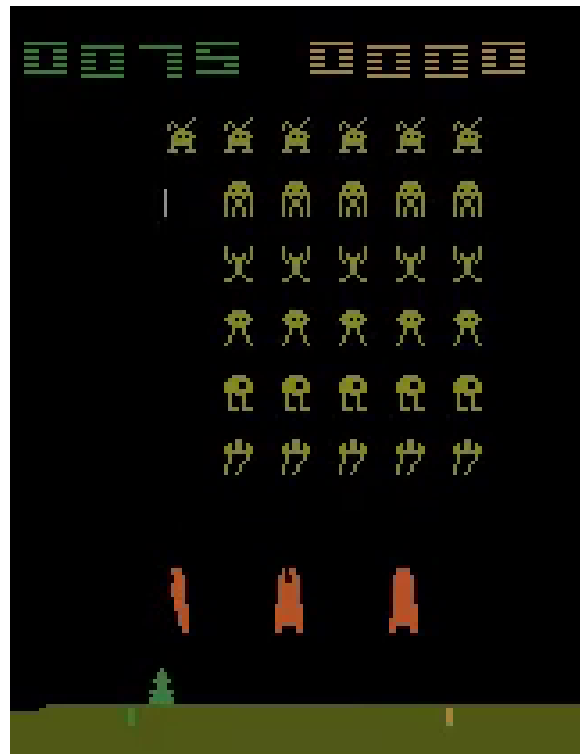
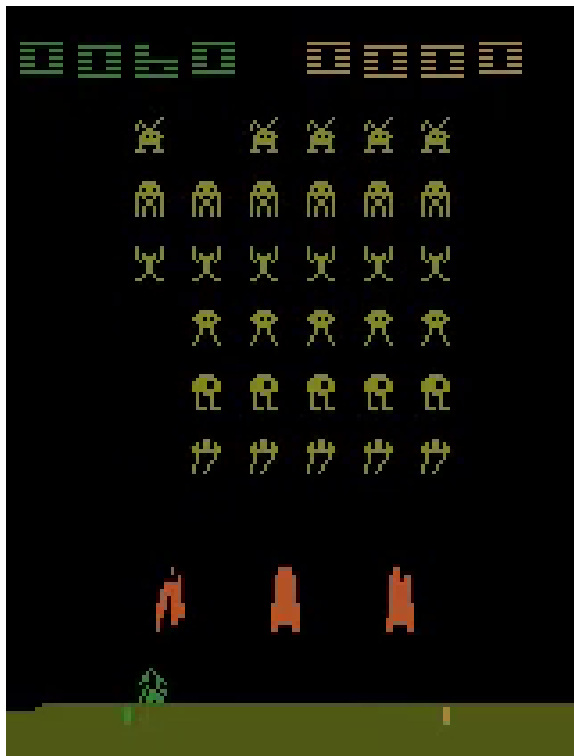


Figura 1. Frame de ejemplo del video grabado: agente aleatorio (izq.) y agente de regla simple (der.) jugando ALE/SpaceInvaders-v5.

Resultados (1 episodio cada uno, generados y grabados con `generar_video_agente`, videos en `notebook/videos/`): el agente aleatorio sobrevivio 449 steps con un return total de 175.0; el agente de regla simple (disparar constantemente mientras alterna movimiento) sobrevivio 607 steps con un return total de 210.0. El agente de regla simple supero al aleatorio porque en Space Invaders la recompensa depende exclusivamente de impactar invasores: agente_aleatorio reparte su probabilidad entre las 6 acciones por igual, asi que solo dispara la mitad de las veces y buena parte de esos disparos ocurre sin que el canon se haya movido a una posicion util; la regla simple siempre esta disparando mientras recorre el eje horizontal, lo que aumenta la probabilidad de que cada disparo impacte algo. Ninguno de los dos agentes fue entrenado: la diferencia viene enteramente de la heuristica incorporada, igual que con la politica simple de CartPole-v1 en el laboratorio anterior.